

Motif patterns in 2D

Alberto Apostolico^{a,b}, Laxmi Parida^c, Simona E. Rombo^{d,*}

^a College of Computing, Georgia Institute of Technology, 801 Atlantic Dr., Atlanta 30332, GA, USA

^b Department of Information Engineering, University of Padua, via Gradenigo, 6/A, I-35131 Padova, Italy

^c IBM T.J. Watson Research Center, Yorktown Heights, NY 10598, USA

^d DEIS, University of Calabria, via P. Bucci, 87036 Arcavacata di Rende, CS, Italy

Received 20 June 2006; received in revised form 17 July 2007; accepted 6 October 2007

Communicated by M. Crochemore

Abstract

Motif patterns consisting of sequences of intermixed solid and don't-care characters have been introduced and studied in connection with pattern discovery problems of computational biology and other domains. In order to alleviate the exponential growth of such motifs, notions of maximal saturation and irredundancy have been formulated, whereby more or less compact subsets of the set of all motifs can be extracted, that are capable of expressing all others by suitable combinations. In this paper, we introduce the notion of maximal irredundant motifs in a two-dimensional array and develop initial properties and a combinatorial argument that poses a linear bound on the total number of such motifs. The remainder of the paper presents approaches to the discovery of irredundant motifs both by offline and incremental algorithms.

© 2007 Elsevier B.V. All rights reserved.

Keywords: Design and analysis of algorithms; Pattern matching; Pattern discovery; Motif; 2D irredundant motifs

1. Introduction

The extraction of motif patterns consisting of intermixed solid and don't care characters from sequences has been widely studied in molecular biology and other domains (cf., e.g. [19,17,11]). Perhaps the single most challenging task in this problem is posed by the number of candidate motifs, which typically grows exponentially with the size of the input. Notions of saturation and irredundancy have been introduced in order to circumscribe such a growth (see, e.g. [6,15]) by producing small subsets of motifs that are capable of expressing all the others.

Apart for pattern discovery, string patterns with don't cares have displayed interesting performances in the dual domain of data compression [7], particularly in cases where the don't cares can be interpolated at the receiver with negligible loss, like in dealing with signals and images. Motivated in part by this, we deal in the present paper with the problem of extracting motif patterns from two dimensional arrays, upon developing suitable extensions of the notions of saturation and irredundancy previously applied to strings. This represents, to the best of our knowledge,

* Corresponding author. Tel.: +39 0984 494774.

E-mail addresses: axa@cc.gatech.edu (A. Apostolico), parida@us.ibm.com (L. Parida), simona.rombo@deis.unical.it (S.E. Rombo).

the first attempt in this direction. In particular, we study the extraction of compact subsets of motifs, called *bases*, the elements of which are capable of expressing all others by suitable combinations. Such motifs originate from the *autocorrelation* of a two-dimensional image, representing intuitively how much the image matches itself. We then develop a combinatorial argument that poses a linear bound on the size of such base. We discuss three approaches to the two-dimensional base extraction. The first one rests on an extension of a classical algorithm by Fisher & Paterson [13]. This approach has good asymptotic complexity, however, it needs to resort to the FFT computation, which is hardly practical with large inputs. The other two algorithms are slower but also more viable. Specifically, our second approach is, like the first, an offline construction that exploits both quadrees [18] and two-dimensional dictionary matching [4], while the third constitutes an online incremental construction of the base.

The paper is organized as follows. Section 2 is devoted to recapturing basic definitions and properties and introducing extensions and novel notions, with various examples and illustrations. Section 3 introduces to the the problem of extracting bases of two dimensional motifs and presents the first two approaches. Section 4 is devoted to the description of the incremental construction, of which the algorithmic implementation is discussed in Section 5. Section 6 contains some concluding remarks.

2. Properties

Let

$$I_{[m,n]} = \left\| \begin{array}{cccc} i_{11} & i_{12} & \dots & i_{1n} \\ i_{21} & i_{22} & \dots & i_{2n} \\ \vdots & & & \\ \vdots & & & \\ i_{m1} & i_{m2} & \dots & i_{mn} \end{array} \right\|$$

be a rectangular array (hereafter, *image*) of $m \times n$ characters over an alphabet Σ . With reference to any rectangular subarray P of I , the *border* of P is defined in a natural way as the set of *edges* delimiting (the smallest rectangle that encloses) P on each of its 4 *sides*. The *size* of a side is the number of characters directly adjacent to its corresponding edge. Hence, the size of the border or *perimeter* coincides with that of the array, except for the two degenerate cases where two edges have size one, so that the size of the border is twice the size of the array. A character of P that is not adjacent to an edge of P is *internal*, otherwise it is *external*.

In addition to the characters from Σ , called *solid* characters, we also introduce a special character denoted by ‘.’ and called a “don’t care”. We use:

$$S_{i,j} = \left\| \begin{array}{cccc} i_{ij} & i_{ij+1} & \dots & i_{in} \\ \vdots & & & \\ \vdots & & & \\ \vdots & & & \\ i_{mj} & i_{mj+1} & \dots & i_{mn} \end{array} \right\|$$

to denote the *bite* of I that starts at position (i, j) .

The following definitions are recaptured to deal with the problem of extracting motifs in two dimensions. In some cases, they match or extend the corresponding one-dimensional ones in [6].

Definition 1 ($\sigma_1 <, =, \leq \sigma_2$). If σ_1 is a don’t care character then $\sigma_1 < \sigma_2$. If both σ_1 and σ_2 are identical characters in Σ , then $\sigma_1 = \sigma_2$. If either $\sigma_1 < \sigma_2$ or $\sigma_1 = \sigma_2$ holds, then $\sigma_1 \leq \sigma_2$.

Definition 2 (P Occurs at $[k, l]$, Cover). An image P of size $m_p \times n_p$ on $\Sigma \cup \{‘.’\}$, with $m_p \leq m$ and $n_p \leq n$, occurs at position $[k, l]$ ($k \leq m - m_p + 1$ and $l \leq n - n_p + 1$) in I if $P[i, j] \leq I[k + i - 1, l + j - 1]$ holds for $1 \leq i \leq m_p$ and $1 \leq j \leq n_p$. Image P is said to *cover* the rectangular interval $[k, l] \times [k + m_p - 1, l + n_p - 1]$ on I , individuated by its high-left and low-right opposite corners.

Example 1. Consider the image P in Fig. 1; it occurs at position $[2, 3]$ in I , covering the rectangular interval $[2, 3] \times [4, 5]$.

$$I_{[4,5]} = \begin{vmatrix} a & b & b & a & a \\ b & a & b & b & a \\ a & b & a & b & b \\ a & a & b & b & b \end{vmatrix} \quad P_{[3,3]} = \begin{vmatrix} b & \cdot & a \\ a & b & b \\ b & b & b \end{vmatrix}$$

Fig. 1. The image P occurs at position $[2, 3]$ in I .

Definition 3 (Two Dimensional Motif M , Location List $\mathcal{L}_M$). Given an image I on the alphabet Σ and a positive integer k , with $k \leq m \times n$, an image M on $\Sigma \cup \{\cdot, \cdot\}$ is a k -motif of I with location list $\mathcal{L}_M = (l_1, l_2, \dots, l_p)$, where each l is a pair of indices of I , if all of the following hold: (1) there is at least one solid character adjacent to each edge of M (2) $p \geq k$, and (3) there is no location $l, l \neq l_i, 1 \leq i \leq p$ such that M occurs on I (the location list is of maximal size).

The first condition ensures that at least one of the external characters on each edge of the motif is solid; otherwise, the motif could have an arbitrary number of external rows and columns made of don't care, conveying no extra information. The third condition ensures that any two distinct location lists must correspond to distinct motifs.

Example 2. With reference to the image I in Fig. 1, consider the two arrays:

$$M'_{[2,3]} = \begin{vmatrix} a & b & b \\ b & \cdot & b \end{vmatrix} \quad M''_{[2,3]} = \begin{vmatrix} a & \cdot & \cdot \\ \cdot & \cdot & b \end{vmatrix}$$

with location lists, respectively, $\mathcal{L}_{M'} = ([1, 1], [2, 2], [3, 3])$ and $\mathcal{L}_{M''} = ([1, 1], [2, 2], [3, 1], [3, 3])$; they are both 3-motifs, but only M'' is also a 4-motif.

In the following we omit the specification of k and speak simply of motif when this causes no confusion (in general, we refer to 2-motifs if not differently specified).

Definition 4 ($M_1 \leq M_2$). Given two motifs M_1 and M_2 of respective sizes $m_1 \times n_1$, and $m_2 \times n_2$, with $m_1 \leq m_2$ and $n_1 \leq n_2$, $M_1 \leq M_2$ holds if $M_1[i, j] \leq M_2[i + c, j + d]$ for $1 \leq i \leq m_1$ and $1 \leq j \leq n_1$, for some fixed c and d , with $0 \leq c \leq m_2 - m_1$ and $0 \leq d \leq n_2 - n_1$.

We also say in this case that M_1 is a *sub-motif* of M_2 , and that M_2 *implies* or *extends* or *covers* M_1 .

Example 3. The motif $M_{[2,2]} = \begin{vmatrix} b & \cdot \\ \cdot & b \end{vmatrix}$ is a sub-motif of the motif M' in Example 2, for $c = 0$ and $d = 1$.

Definition 5 (Maximal Motif, Box). Let $M_1, M_2, \dots, M_f$ be the motifs in an image I . A motif M_i is *maximal in composition* if and only if there exist no $M_l, l \neq i$, with $\mathcal{L}_{M_i} = \mathcal{L}_{M_l}$ and $M_i \leq M_l$. A motif M_i , maximal in composition, is also *maximal in length* if and only if there exists no motif $M_j, j \neq i$, such that M_i is a submotif of M_j and $|\mathcal{L}_{M_i}| = |\mathcal{L}_{M_j}|$. A *maximal motif* is maximal both in composition and in length. A maximal motif with no don't care is called a *box*.

Example 4. Consider the two motifs:

$$M_{1[2,3]} = \begin{vmatrix} a & \cdot & b \\ b & \cdot & b \end{vmatrix} \quad M_{2[2,2]} = \begin{vmatrix} b & b \\ \cdot & b \end{vmatrix}$$

with $\mathcal{L}_{M_1} = ([1, 1], [2, 2], [3, 3])$ and $\mathcal{L}_{M_2} = ([1, 2], [2, 3], [3, 4])$ on the image I in Fig. 1. The motif M_1 is not maximal in composition, since $\mathcal{L}_{M_1} = \mathcal{L}_{M'}$ and $M_1 \leq M'$, where M' is the the first motif in Example 2. At the same time, the motif M_2 is not maximal in length, since $|\mathcal{L}_{M_2}| = |\mathcal{L}_{M'}|$ and $M_2 \leq M'$. The motif M' is maximal both in composition and in length, as well as its submotif $\begin{vmatrix} b & \cdot \\ \cdot & b \end{vmatrix}$, having $\mathcal{L}_M = ([1, 2], [1, 3], [2, 1], [2, 3], [2, 4], [3, 2], [3, 4])$.

Example 5. An example of box of the image I in Fig. 1 is the motif $M_{[2,2]} = \begin{vmatrix} b & a \\ a & b \end{vmatrix}$, with $\mathcal{L}_M = ([1, 2], [3, 2])$.

Definition 6 (*Straddle*). Two sets $\mathcal{L}_1$ and $\mathcal{L}_2$ *straddle* if $\mathcal{L}_1 \setminus \mathcal{L}_2 \neq \emptyset$ and $\mathcal{L}_2 \setminus \mathcal{L}_1 \neq \emptyset$.

The last definition allows us to focus on an important property of boxes, captured by the following lemma that extends a result in [10].

Lemma 1. In any $m \times n$ image I on an alphabet Σ , there are at most mn boxes.

Proof. Given an $m \times n$ image I and an integer $k \geq 2$, consider all maximal k -motifs that consist of boxes. Then any two distinct maximal motifs M_1 and M_2 are such that only one of the following holds: (1) $\mathcal{L}_{M_1} \cap \mathcal{L}_{M_2} = \emptyset$, or (2) without loss of generality, $\mathcal{L}_{M_1} \subset \mathcal{L}_{M_2}$. Let $\mathbf{L}$ be the collection of all the location lists of such motifs. Since no two location lists straddle, a hierarchical tree can be built such that (1) each leaf in the tree corresponds to a position on I (an element of the location list which in this case is a pair (i, j)) and (2) each $\mathcal{L} \in \mathbf{L}$ corresponds to an internal node such that the set of all the leaves reachable from this internal node is exactly $\mathcal{L}$. Now, the number of leaf nodes is no more than mn and the number of internal nodes is no more than the leaf nodes ($\leq mn$). Hence $|\mathbf{L}| \leq mn$. Thus the number of boxes is bounded by mn . ■

When maximal motifs with don't care are considered, their number can increase exponentially. Furthermore, the information carried out by some of these motifs can turn out to be redundant, being already expressed by other motifs. The following definitions isolate a particular class of motifs, that enjoy the property that their information content is guaranteed not to be redundant.

Definition 7 (*Redundant Motif*). A maximal motif M in I , with location list $\mathcal{L}_M$, is *redundant* if there exist maximal submotifs M_i , $1 \leq i \leq p$, such that $\mathcal{L}_M = \mathcal{L}_{M_1} \cup \mathcal{L}_{M_2} \cup \dots \cup \mathcal{L}_{M_p}$, up to some offsets, and $M \preceq M_i$, $1 \leq i \leq p$ (i.e. every occurrence of M on I is already covered by one of the motifs $M_1, M_2, \dots, M_p$). M is also said to be *implied* by $M_1, M_2, \dots, M_p$.

Definition 8 (*Irredundant Motif*). A maximal motif that is not redundant is called an *irredundant* motif.

Example 6. Consider the following image I and its three motifs M_1, M_2 and M_3 :

$$I_{[4,5]} = \begin{vmatrix} b & a & b & a & b \\ a & b & b & a & a \\ b & b & a & b & a \\ b & b & a & b & a \end{vmatrix} \quad M_{1[2,3]} = \begin{vmatrix} a & b & \cdot \\ \cdot & b & a \end{vmatrix}$$

$$M_{2[2,3]} = \begin{vmatrix} a & b & a \\ \cdot & b & a \end{vmatrix} \quad M_{3[2,3]} = \begin{vmatrix} a & b & \cdot \\ b & b & a \end{vmatrix}$$

where $\mathcal{L}_{M_1} = ([1, 2], [2, 1], [3, 3])$, $\mathcal{L}_{M_2} = ([1, 2], [3, 3])$ and $\mathcal{L}_{M_3} = ([1, 2], [2, 1])$. Thus, M_1 is implied by M_2 and M_3 that are, on the contrary, irredundant.

Definition 9 (*Base*). Given an image I on an alphabet Σ , let $\mathcal{M}$ be the set of all maximal motifs on I . A subset $\mathcal{B}$ of $\mathcal{M}$ is called a *base* of $\mathcal{M}$ if the following hold: (1) for each $M \in \mathcal{B}$, M is irredundant with respect to $\mathcal{B} - \{M\}$, and, (2) let $G(\mathcal{X})$ be the set of all the redundant maximal motifs implied by the set of motifs $\mathcal{X}$: then $\mathcal{M} = G(\mathcal{B})$.

Thus, a base is a set of motifs that are irredundant and able to generate all the other maximal motifs in the given image.

Example 7. Consider the image:

$$I_{[4,3]} = \begin{vmatrix} a & a & b \\ b & b & b \\ a & a & b \\ b & a & b \end{vmatrix}.$$

Thus, the following motifs are a base for I :

$$\begin{aligned}
M_{1[1,2]} &= \left\| \begin{array}{cc} a & b \end{array} \right\| & \mathcal{L}_{M_1} &= ([1, 2], [3, 2], [4, 2]) \\
M_{2[2,2]} &= \left\| \begin{array}{cc} a & b \\ \cdot & b \end{array} \right\| & \mathcal{L}_{M_2} &= ([1, 2], [3, 2]) \\
M_{3[2,2]} &= \left\| \begin{array}{cc} b & b \\ a & \cdot \end{array} \right\| & \mathcal{L}_{M_3} &= ([2, 1], [2, 2]) \\
M_{4[2,2]} &= \left\| \begin{array}{cc} \cdot & b \\ a & b \end{array} \right\| & \mathcal{L}_{M_4} &= ([2, 2], [3, 2]) \\
M_{5[2,3]} &= \left\| \begin{array}{ccc} a & a & b \\ b & \cdot & b \end{array} \right\| & \mathcal{L}_{M_5} &= ([1, 1], [3, 1]) \\
M_{6[3,2]} &= \left\| \begin{array}{cc} a & \cdot \\ b & b \\ a & \cdot \end{array} \right\| & \mathcal{L}_{M_6} &= ([1, 1], [1, 2]) \\
M_{7[3,1]} &= \left\| \begin{array}{c} b \\ \cdot \\ b \end{array} \right\| & \mathcal{L}_{M_7} &= ([1, 3], [2, 1], [2, 3]).
\end{aligned}$$

In particular, the motif $\left\| \begin{array}{cc} \cdot & b \\ a & \cdot \end{array} \right\|$ is, for example, implied by M_3 and M_4 .

Definition 10 (*Consensus and Meet*). Given two images I_1 and I_2 on $\Sigma \cup \{', '\}$ with $size(I_1) = m_1 \times n_1$, $size(I_2) = m_2 \times n_2$, the *consensus* of I_1 and I_2 is the image $C_{1,2}$ such that $size(C_{1,2}) = m_c \times n_c$, where $m_c = \min\{m_1, m_2\}$ and $n_c = \min\{n_1, n_2\}$, and defined as: $C_{1,2}[i, j] = I_1[i, j]$ if $I_1[i, j] = I_2[i, j]$ and $C_{1,2}[i, j] = \cdot$ otherwise ($i = 1, 2, \dots, m_c$, $j = 1, 2, \dots, n_c$). Deleting from $C_{1,2}$ all the external rows and columns made up of only don't care symbols yields an (possibly empty) image that is called the *meet* of I_1 and I_2 and is denoted by $M_{1,2}$.

In words, the consensus between two images illustrates how much they match, which is represented by the density of solid symbols it contains; the meet conveys the same information content of the consensus, but it is cleaned of the peripheral, non-informative rows and columns.

Example 8. Consider the following two images I_1 and I_2 :

$$I_{1[4,4]} = \left\| \begin{array}{cccc} a & a & b & b \\ b & a & a & a \\ a & b & b & b \\ a & b & b & a \end{array} \right\| \quad I_{2[4,3]} = \left\| \begin{array}{ccc} b & a & a \\ a & a & a \\ b & a & b \\ b & a & a \end{array} \right\|.$$

Thus, their consensus $C_{1,2}$ and their meet $M_{1,2}$ are, respectively:

$$C_{1,2[4,3]} = \left\| \begin{array}{ccc} \cdot & a & \cdot \\ \cdot & a & a \\ \cdot & \cdot & b \\ \cdot & \cdot & \cdot \end{array} \right\| \quad M_{1,2[3,2]} = \left\| \begin{array}{cc} a & \cdot \\ a & a \\ \cdot & b \end{array} \right\|.$$

A crucial step in the extraction of the base from an image I is computing how much a displaced copy of I matches I itself at each displacement. This is only in part exhausted with computing the meets between I and each one of its bites. Consider the following image I and its motif M :

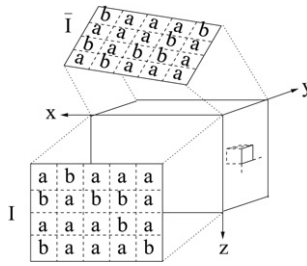


Fig. 2. A geometrical interpretation of an image I , and the transposition of it and of its rotation $\bar{I}$ on orthogonal faces of a parallelepiped.

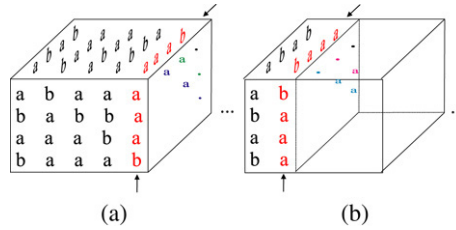


Fig. 3. Filling the cells of the parallelepiped from (a) the fifth column of I and the fifth column of $\bar{I}$; (b) the second column of I and the second column of $\bar{I}$.

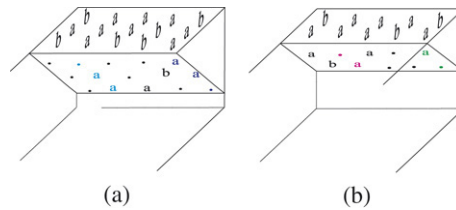


Fig. 4. Diagonal planes of autocorrelation corresponding to the consensus between I and (a) Bite $S_{2,1}$; (b) Bite $S_{3,1}$.

$$I_{[3,4]} = \begin{vmatrix} a & b & a & b \\ b & a & b & b \\ a & a & b & a \\ b & a & a & a \end{vmatrix}$$

$$M_{[2,3]} = \begin{vmatrix} b & a & b \\ a & b & . \end{vmatrix}$$

$$\bar{I}_{[3,4]} = \begin{vmatrix} a & b & a & a \\ b & a & b & b \\ a & b & a & b \end{vmatrix}$$

$$\hat{A}_{2,2[2,3]} = \begin{vmatrix} a & b & . \\ b & a & b \end{vmatrix}.$$

with $\mathcal{L}_M = ([1, 2], [2, 1])$. It is easy to see that no meet between I and its bites can generate M . For this, we need to rotate I upside down and consider the meet of the rotated image $\bar{I}$ with its bite $S_{2,2}$, denoted by $\hat{A}_{2,2}$. Then, the motif M of I can be obtained by rotating $\hat{A}_{2,2}$ upside down.

Definition 11 (Autocorrelation). Given an image I on $\Sigma \cup \{.\}$, let $\bar{I}$ be the image obtained by rotating I upside down while retaining the original enumeration of rows and columns. We use the term *autocorrelations* of I referring to the collection of the meets generated by either I or $\bar{I}$ with their respective bites (with the forethought of rotating upside down those meets coming from $\bar{I}$). The autocorrelation generated by I and $S_{i,j}$ is denoted by $A_{i,j}$, whereas $\bar{A}_{i,j}$ denotes the autocorrelation generated symmetrically on $\bar{I}$.

Figs. 2–5 give graphical representations of both an image I and its autocorrelations. In particular, to provide a geometrical interpretation of an image I of size $m \times n$, we can consider a rectangular polygon made of $m \times n$ squares having sides of unitary size. Thus, a generic element occurring at the position $[i, j]$ of I is contained in the square whose corners have coordinates $(i - 1, j - 1)$, $(i, j - 1)$, $(i - 1, j)$, and (i, j) in the real plane $\mathbb{R}^2$ (see also [3]). Consider now a parallelepiped in the real space $\mathbb{R}^3$ and write the image I on one of its vertical face, the rotated image

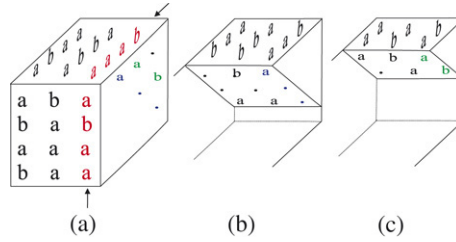


Fig. 5. (a) The parallelepiped originating from different portions of I (the first three columns) and of $\bar{I}$ (the last three columns). The figures on the right display the autocorrelations corresponding to the meets between I and Bite $S_{2,3}$ (b), and I and Bite $S_{3,3}$ (c).

$\bar{I}$ on the adjacent upper horizontal face, as illustrated in Fig. 2. Next, fill the cells of the parallelepiped in analogy to the one-dimensional case considered in [6], as follows. For each j along the x axis ($j = 1 \dots n$), consider the corresponding columns of I and $\bar{I}$, respectively, and compare the i -th element along z ($i = 1 \dots m$) of such column of I with the element $i + 1$ along y of the corresponding column of $\bar{I}$. As an example, in Fig. 3 (a) the last columns (pointed out by an arrow) of both I and $\bar{I}$ are compared, while their second columns in Fig. 3 (b). If the compared elements match, then the matching character is inserted in the cell of the parallelepiped corresponding to the position $[j, i + 1, i]$, according to the same convention adopted for images in $\mathbb{R}^2$. Otherwise, the symbol $\cdot$ is inserted in that cell.

Once the parallelepiped is filled in this way, slicing it into diagonal planes as in Fig. 4 yields the consensus between I and some of its bites, from which it is possible to extract m of the $O(N)$ autocorrelations. For a graphical representation of all autocorrelations, one would have to apply the same treatment to different portions of I and of $\bar{I}$ (see Fig. 5). Summarizing, we obtain n parallelepipeds; each parallelepiped k ($k = 1 \dots n$) contains the m autocorrelations coming from the meets between I and its meets $S_{h,k}$ ($h = 1 \dots m$). By reversing I and $\bar{I}$ and applying the same reasoning, the remaining autocorrelations are produced.

As claimed in the following theorem, the set of irredundant 2-motifs of an image I is a subset of all the autocorrelations of I .

Theorem 1. *Given an image I on an alphabet Σ , let $\mathcal{A}$ be the set of the autocorrelations of I and $\mathcal{I}$ the set of all the irredundant 2-motifs of I . Then $\mathcal{I} \subseteq \mathcal{A}$.*

Proof. Let M be a generic element of $\mathcal{I}$ and suppose that $M \notin \mathcal{A}$. Then, every occurrence of M must be contained in some meet of I and one of its bites. Let $\mathcal{M} \subseteq \mathcal{A}$ be the set of all such meets. Since each motif of $\mathcal{M}$ contains some occurrences of M , and $\mathcal{M}$ covers all the occurrences of M , then, up to some offsets, $\mathcal{L}_M \equiv \cup \mathcal{L}_{M_i}$, $M_i \in \mathcal{M}$, which contradicts the hypothesis that M is irredundant. ■

Theorem 2. *Let $\mathcal{B}$ be a base of 2-motifs for the image I on an alphabet Σ , and $\mathcal{A}$ the set of the autocorrelations of I , then $\mathcal{B} \subseteq \mathcal{A}$.*

Proof. This follows from Theorem 1 as a corollary. In fact, by Theorem 1 we know that all the irredundant 2-motifs of I are in $\mathcal{A}$. So, if a base $\mathcal{B}$ exists, $\mathcal{B}$ must be contained in $\mathcal{A}$.

First, we prove that it is possible to build a base for all the elements of $\mathcal{A}$. Following that, we will see that such a base generates all the 2-motifs of I .

Consider two sets $\mathcal{T}$ and $\mathcal{U}$. Let at the beginning $\mathcal{T} \equiv \mathcal{A}$ and $\mathcal{U} \equiv \emptyset$. Choose an arbitrary motif $M_j \in \mathcal{T}$, with the forethought of checking only once each motif in $\mathcal{T}$, and search for the motifs $M_i \in \mathcal{T} - \{M_j\}$ such that $M_j \preceq M_i$ and $\mathcal{L}_{M_i} \subseteq \mathcal{L}_{M_j}$. Every time we find a new M_i , we put it in $\mathcal{U}$. If it happens that $\cup \mathcal{L}_{M_i} \equiv \mathcal{L}_{M_j}$, $M_i \in \mathcal{U}$, then $\mathcal{T} = \mathcal{T} - M_j$. At the end, we have that $\mathcal{T}$ contains all the motifs of $\mathcal{A}$ that are not expressible by some other motifs in $\mathcal{T}$ and $\mathcal{A} - \mathcal{T}$ contains all the redundant motifs of $\mathcal{A}$ generated by the motifs of $\mathcal{T}$. So we can say that $\mathcal{T}$ is a base for $\mathcal{A}$.

Now we have to prove that all the 2-motifs of I can be generated by the motifs of $\mathcal{T}$.

Consider a generic motif M of I such that $M \notin \mathcal{A}$. Then, for each one of its occurrences there will be a meet of I and some of its bites that contain such an occurrence. Therefore, we can say that M is generated by some subset $\mathcal{S}$ of $\mathcal{A}$, since $\mathcal{L}_M = \cup \mathcal{L}_{M_i}$, $M \preceq M_i$, $M_i \in \mathcal{S}$. But if $\mathcal{S} \subseteq \mathcal{A}$ then every motif of $\mathcal{S}$ is generated from the base $\mathcal{T}$ of $\mathcal{A}$. This means that, $\forall M_i \in \mathcal{S}$, $\mathcal{L}_{M_i} = \cup \mathcal{L}_{M_j}$ and $M_i \preceq M_j$, $M_j \in \mathcal{T}$. As a consequence $\mathcal{L}_M = \cup \mathcal{L}_{M_j}$, $M_j \preceq M$, $M_j \in \mathcal{T}$, and $\mathcal{T} = \mathcal{B}$. ■

Irrespective of dimension and of the value of k , it is natural to ask whether the base is unique and, if not, whether the cardinality of a base is unique. The following theorem answers both in the affirmative, based on an argument that holds for any dimension:

Theorem 3. *The base $\mathcal{B}$ of k -motifs for the image I on an alphabet Σ is unique for any k .*

Proof. For any fixed k , if there are no k -motifs then clearly the base is empty as well. Assuming that some k -motif exists, let $h \geq k$ be the smallest integer such that some k -motif has precisely h occurrences. We claim that every motif with this property must be in the base. Otherwise, its list could be aggregated by taking the union of lists of other, supposedly irredundant motifs. But this would contradict the assumed minimality of h : since the tributaries have size h and non-empty intersections, then the combined list must have size at least $h + 1$. We now consider the next value $h' > h$, if it exists, for which some k -motif has precisely h' occurrences. Clearly, any such motif may be redundant only on account of the motifs, considered earlier, that have precisely h occurrences. Otherwise, it must be irredundant and must be added to the base. Continuing in this fashion with increasing values of h proves the claim. ■

Corollary 1. *The base of k -motifs for the image I on an alphabet Σ is the set of irredundant k -motifs of I .*

Proof. It follows from Theorem 3. ■

3. Extracting the base

We now address the extraction of the base of 2-motifs in an image I of size $N = m \times n$, where we assume w.l.o.g. $m \leq n$. A straightforward approach would consist of three main steps: the first step extracts the set $\mathcal{A}$ of the autocorrelations of I , the second step computes $\mathcal{L}_M$ for each $M \in \mathcal{A}$, and, finally, the third step discards all the redundant motifs in $\mathcal{A}$, leaving only the base for I . As seen next, the first and third steps have each a cost of $O(N^2)$.

In order to build $\mathcal{A}$, we have first to extract all the autocorrelations and then to delete possible duplicates. Since the number of bites is linear in N , and the time for computing each autocorrelation is $O(N)$, then the cost of the first part is $O(N^2)$. The second part is done by inserting each autocorrelation in succession into a trie, say, in row-major order, and checking for duplicates. Clearly, also the cumulative cost of this part is $O(N^2)$. As for the third step, let $\mathbf{L}_{\mathcal{A}}$ be the collection of the location lists of all the motifs in $\mathcal{A}$. For each $\mathcal{L}_M \in \mathbf{L}_{\mathcal{A}}$, if $\mathcal{L}_M = \mathcal{L}_{M_1} \cup \mathcal{L}_{M_2} \dots \cup \mathcal{L}_{M_h}$ up to some offsets, with $M_1, \dots, M_h \in \mathcal{A}$ and $M_i \neq M$ ($i = 1, 2, \dots, h$), then M is redundant. If, on the other hand, there is no way to express $\mathcal{L}_M$ by other location lists in $\mathbf{L}_{\mathcal{A}}$, then M is irredundant and we can add it to the output set. This step is afforded in $O(N)$ time for each list (cf., e.g., [15]) by checking to see whether all occurrences in $\mathcal{L}_M$ falls into the “footprints” of some occurrence of some of the other motifs. We have now to analyse the second step, i.e. the computation of the list of occurrences for each autocorrelation. This constitutes a pattern matching problem with don’t care in two dimensions, which can be reduced to the corresponding one-dimensional problem by converting again each motif to a string row- or column-wise and then running the classical Fischer–Paterson algorithm based on convolution [13] (cf. also the more recent improvements in [12]) on a similarly linearized version of the image. More precisely, to find all the occurrences of a pattern P of size $m_p \times n_p$ in a text T of size $m_t \times n_t$, the conversion to one dimension works as follows. Let t be the string obtained by juxtaposing the m_t rows of T . We have that $|t| = m_t n_t$. Now, append $n_t - n_p$ ($n_p \leq n_t$) of don’t-care symbols to the first $m_p - 1$ rows of P and concatenate all rows in a string p . Clearly, searching for p in t is equivalent to searching for P in T . The application of Fischer–Paterson’s algorithm to p and t takes time $O(m_t n_t \log^2 n_t \log \log n_t)$. Since the number of motifs in $\mathcal{A}$ is at most N , the cost of the second step is thus $O(N^2 \log^2 n \log \log n)$, which is also the dominant term in the time complexity of this approach.

As stated in [13], pattern matching by FFT may be hardly viable, whence an interesting challenge is trying to solve the second step in a more practical way. The remainder of this paper is devoted to illustrate some alternate techniques for this purpose.

One such construction may be based on Quadrees [18] and Dictionary Matching [4], as follows. Let A be one of the $2N$ autocorrelations, i.e., an image on $\Sigma \cup \{‘.’\}$. The main problem is finding all the occurrences of A in the input image I , in order to build the list of occurrences $\mathcal{L}_A$. The image A is stored in a quadtree Q , of which the leaves represent each a maximal block of size a power of 2 and consisting entirely either of don’t cares or of symbols of Σ . These latter blocks will be called *solid* (see Fig. 6).

Let D_A be the dictionary containing all solid blocks in I . Searching for D_A in I represents a two-dimensional dictionary matching problem such as solved, e.g. by the algorithm in [4].

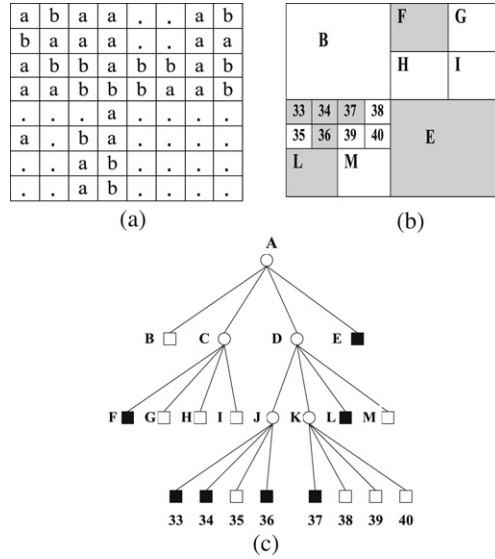


Fig. 6. (a) An autocorrelation A. (b) Blocks in the autocorrelation A. (c) Quadtree representation of A.

Lemma 2. *Computing the occurrences in I of all the solid leaves of the quadtree of an autocorrelation A takes time $O(N \log N)$.*

Proof. Using the construction in [4], for each location $T[i, j]$ of a two-dimensional text T , finding the largest pattern P_i of a dictionary $D = \{P_1, P_2, \dots, P_k\}$ with matches at that location requires $O(d(\log k + \log \sigma_D))$ time for preprocessing the dictionary and $O(t(\log k + \log \sigma_D))$ time for scanning the text, where $d = \sum_{P_i \in D} p_i$, p_i is the size of P_i , t is the size of T and σ_D is the number of distinct characters occurring in D . In the worst case A contains $\Theta(N)$ solid blocks of which the cumulative size cannot exceed the size of A , hence is $O(N)$. Thus, the cost of the preprocessing is $O(N \log N)$. Since the size of I is N , also the text scanning is bounded by $O(N \log N)$. Finally, computing the occurrences of all non-maximal patterns is also done in $O(N \log N)$: any such pattern will be included in one of the largest ones reported by the algorithm and any of the latter cannot contain more than $\log N$ smaller patterns, by the structure of the quadtree. Hence there is a maximum of $O(N \log N)$ occurrences of all patterns in D_A and they are retrieved in $O(N \log N)$ time from the occurrences of maximal patterns and the structure of D_A . ■

At this point, for each position $[i, j]$ of the image I we know the list of the solid blocks of A occurring at such a position.

Lemma 3. *Computing the list $\mathcal{L}_A$ of the occurrences in I of an autocorrelation A takes time $O(N^2)$.*

Proof. To every position $[i, j]$ of I , assign a bit-array indexed to the solid blocks of A . This is done trivially by expanding the lists of occurrences produced during the preprocessing. At this point, checking any position $[i, j]$ of I for the occurrence at $[i, j]$ of any preassigned solid block of A takes constant time. Once all occurrences of this block have been determined, testing each for completion into an occurrence of A takes $O(N)$ time from knowing the block structure of A . Therefore, the overall cost is $O(N^2)$. ■

Theorem 4. *The cost of computing the base $\mathcal{B}$ of the irredundant motifs of an image I using the approach based on quadtree and dictionary matching is $O(N^3)$.*

Proof. The number of autocorrelations in an image I of size N is $O(N)$, and the most time consuming step in the approach described above is just the computation of $\mathcal{L}_A$ for all the autocorrelations. ■

4. Incremental construction

In this section, we describe an approach to the discovery of the base of irredundant motifs for an image I which is based on an incremental construction. The idea is to scan the image bottom up in some fashion, e.g., from right to left

row-wise, and iteratively update the computation of the base relative to the covered region. At the generic step, the scanned region will consist of a rectangular section at the bottom of the image, topped by a (possibly empty) suffix of the highest row in the region. The typical update consists of expanding this suffix by adding the character to its left. If $I[i, j + 1]$ is currently the leftmost character in this suffix, the scanned region will be denoted by $R_{i,j+1}$. Let $\mathcal{B}_{ij}$ be the base of irredundant motifs in R_{ij} . Assuming that we know the base $\mathcal{B}_{i,j+1}$ in $R_{i,j+1}$, we are interested in computing the new base $\mathcal{B}_{ij}$ that results from the expansion of $R_{i,j+1}$ into R_{ij} .

In the transition from $R_{i,j+1}$ to R_{ij} , the repertoire of motifs can change in different ways. In particular, there will be in general some new motif M in $\mathcal{B}_{ij}$ that was not found in $\mathcal{B}_{i,j+1}$. On the other hand, some old motif M' of $\mathcal{B}_{i,j+1}$ could become redundant once new motifs of $\mathcal{B}_{ij}$ would extend occurrences of M' never extended before, including its new occurrences in R_{ij} .

We shall say that a motif M has a *maximal* occurrence at $[i, j]$, or *is maximal* at $[i, j]$ if M results from the meet of S_{ij} with some bite of $R_{i,j+1}$. Clearly, only motifs that are maximal at some position in $R_{i,j+1}$ have a chance of being found in $\mathcal{B}_{i,j+1}$. A maximal occurrence of a motif M in R_{ij} that is not contained in the maximal occurrence of another motif is an *exposed* occurrence. Clearly, the set $\mathcal{B}_{ij}$ is constituted by all and only the motifs having *exposed* occurrences in R_{ij} . We partition the motifs to be handled in the transition from $R_{i,j+1}$ to R_{ij} are as follows:

Class I Motifs of R_{ij} belonging to $\mathcal{B}_{i,j+1}$. In plain words, these are the motifs in $\mathcal{B}_{i,j+1}$ completed each by its possible new occurrence at $[i, j]$.

Class II Motifs of R_{ij} that do not belong to $\mathcal{B}_{i,j+1}$ but have a maximal occurrence at $[i, j]$. Hence, these motifs result from the meet of S_{ij} with a bite of $R_{i,j+1}$.

For ease of exposition, we call *new motifs* the motifs in Class II, *old motifs* those in Class I. We now examine these two classes and study mechanisms that *obliterate* motifs, that is, ways in which motifs become redundant. For a motif to be obliterated, all of its previously exposed occurrences must become contained in maximal occurrences of some other motifs.

Note that an old motif M occurring at $[i, j]$ cannot be obliterated by any other motif M' in $\mathcal{B}_{i,j+1}$ not occurring at $[i, j]$.

Lemma 4. *If an old motif M occurring at $[i, j]$ obliterates a motif $M' \in \mathcal{B}_{i,j+1}$, then M' is a submotif of M .*

Proof. If M' has a new occurrence in R_{ij} , then M' conserves its irredundance since some of its occurrences in $R_{i,j+1}$ are not covered by M . Assume then that M' does not have a new occurrence in R_{ij} , and let M'' be the motif resulting from deleting the leftmost column from M . Since M' had not been obliterated by M at a previous stage, this means that the occurrence of M at $[i, j]$ now covers the occurrence of M' that was met last during the scanning. But M being in $\mathcal{B}_{i,j+1}$ implies that M had at least two occurrence already in $R_{i,j+1}$. But one such occurrence is covering the last seen occurrence of M' , whence M' should have been obliterated earlier. The only possibility is that M obliterates M'' , in that it extends the last occurrence of this motif into an occurrence of M at $[i, j]$. ■

Clearly, old motifs without a new occurrence at $[i, j]$ cannot obliterate any new motif, since for that they would need to cover, in particular, its occurrence at $[i, j]$. Therefore, it remains for us to examine how old motifs having an occurrence at $[i, j]$ may obliterate new ones, and how new motifs may obliterate any other. From the new motifs, it is convenient to single out those that are detected at $[i, j]$ for the first time. The motifs in this sub-class have exactly two occurrences and are called *novel*. Clearly no such motif can be obliterated by old motifs.

Lemma 5. *Novel motifs cannot obliterate one another.*

Proof. Assume for a contradiction that the novel motif M is obliterated by the novel motif $M' \neq M$. Since each must have been obtained by the meet of the bite S_{ij} with some other bite of R_{ij} , then necessarily M and M' must be the same motif or one of the two must have at least three occurrences, a contradiction. ■

In conclusion, novel motifs have an exposed occurrence at $[i, j]$ and therefore must be included in $\mathcal{B}_{ij}$. Motifs that are new but not novel cannot obliterate any other motif: on the one hand, such motifs may be (transitively) obliterated by old motifs in $\mathcal{B}_{i,j+1}$; on the other, any motif obliterated by them would have been already so at some past iteration. In intuitive terms, the novel motifs detected at $[i, j]$ expand and subsume previously detected motifs, whereas the rest of the new motifs extract and revive previously obliterated motifs.

Example 9. Consider the following image, with bold symbols indicating the explored region $R_{2,3}$:

$$\begin{array}{c} \begin{array}{ccccc} & 1 & 2 & 3 & 4 & 5 \\ \begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \end{array} & \left\| \begin{array}{ccccc} \mathbf{b} & a & a & b & b \\ a & b & \mathbf{a} & \mathbf{a} & \mathbf{a} \\ \mathbf{b} & \mathbf{b} & a & \mathbf{b} & a \\ \mathbf{b} & a & a & b & a \end{array} \right\| \end{array} \end{array}$$

The base extracted at $[2, 3]$ is $\mathcal{B}_{2,3} = \{M_1, M_2, M_3, M_4, M_5, M_6, M_7, M_8, M_9, M_{10}, M_{11}\}$, where:

$$\begin{array}{ll} M_{1[1,2]} = \left\| \begin{array}{cc} a & a \end{array} \right\| & \mathcal{L}_{M_1} = ([2, 3], [2, 4], [4, 2]) \\ M_{2[1,2]} = \left\| \begin{array}{cc} b & a \end{array} \right\| & \mathcal{L}_{M_2} = ([3, 2], [3, 4], [4, 1], [4, 4]) \\ M_{3[1,3]} = \left\| \begin{array}{ccc} a & \cdot & a \end{array} \right\| & \mathcal{L}_{M_3} = ([2, 3], [3, 3], [4, 3]) \\ M_{4[1,3]} = \left\| \begin{array}{ccc} a & b & a \end{array} \right\| & \mathcal{L}_{M_4} = ([3, 3], [4, 3]) \\ M_{5[1,4]} = \left\| \begin{array}{cccc} b & \cdot & a & b \end{array} \right\| & \mathcal{L}_{M_5} = ([3, 1], [4, 1]) \\ M_{6[3,1]} = \left\| \begin{array}{c} a \\ a \\ a \end{array} \right\| & \mathcal{L}_{M_6} = ([2, 3], [2, 5]) \\ M_{7[2,2]} = \left\| \begin{array}{cc} b & a \\ \cdot & a \end{array} \right\| & \mathcal{L}_{M_7} = ([3, 2], [3, 4]) \\ M_{8[2,2]} = \left\| \begin{array}{cc} b & \cdot \\ b & a \end{array} \right\| & \mathcal{L}_{M_8} = ([3, 1], [3, 4]) \\ M_{9[2,2]} = \left\| \begin{array}{cc} \cdot & a \\ b & a \end{array} \right\| & \mathcal{L}_{M_9} = ([2, 4], [3, 4]) \\ M_{10[2,2]} = \left\| \begin{array}{cc} \cdot & a \\ a & \cdot \end{array} \right\| & \mathcal{L}_{M_{10}} = ([2, 3], [3, 2]) \\ M_{11[2,3]} = \left\| \begin{array}{ccc} a & \cdot & a \\ a & b & a \end{array} \right\| & \mathcal{L}_{M_{11}} = ([2, 3], [3, 3]). \end{array}$$

We are interested in analysing how the base evolves in the transition from $R_{2,3}$ to $R_{2,2}$. All the motifs in $\mathcal{B}_{2,3}$ are old motifs when the region $R_{2,2}$ is considered. In particular, M_2 , M_7 , M_8 and M_9 have a new occurrence at $[2, 2]$. The new motifs are:

$$\begin{array}{ll} M_{12[1,3]} = \left\| \begin{array}{ccc} b & \cdot & a \end{array} \right\| & \mathcal{L}_{M_{12}} = ([2, 2], [3, 1], [4, 1]) \\ M_{13[1,3]} = \left\| \begin{array}{ccc} b & a & a \end{array} \right\| & \mathcal{L}_{M_{13}} = ([2, 2], [4, 1]) \\ M_{14[2,2]} = \left\| \begin{array}{cc} b & a \\ b & a \end{array} \right\| & \mathcal{L}_{M_{14}} = ([2, 2], [3, 4]) \\ M_{15[2,4]} = \left\| \begin{array}{cccc} b & a & \cdot & a \\ \cdot & a & b & a \end{array} \right\| & \mathcal{L}_{M_{15}} = ([2, 2], [3, 2]) \end{array}$$

$$M_{16[2,3]} = \begin{vmatrix} b & \cdot & a \\ b & a & \cdot \end{vmatrix} \quad \mathcal{L}_{M_{16}} = ([2, 2], [3, 1])$$

$$M_{17[3,2]} = \begin{vmatrix} \cdot & a \\ \cdot & \cdot \\ a & \cdot \end{vmatrix} \quad \mathcal{L}_{M_{17}} = ([2, 2], [2, 3])$$

$$M_{18[3,2]} = \begin{vmatrix} \cdot & a \\ b & a \\ \cdot & a \end{vmatrix} \quad \mathcal{L}_{M_{18}} = ([2, 2], [2, 4]).$$

The motif M_{12} is new but not novel: it was a submotif of M_5 before, so that it does not appear in $\mathcal{B}_{2,3}$, but it has an exposed occurrence at $[2, 2]$. All the other new motifs are also novel, and some of them obliterate other old motifs. In particular, M_7 is obliterated by M_{15} and M_{18} ; M_8 by M_{14} and M_{16} ; M_9 by M_{14} and M_{18} . Furthermore, M_{10} is a sub-motif of M_{16} , M_{11} is a submotif of M_{15} and M_6 is a sub-motif of M_{18} . Thus, the new base is:

$$\mathcal{B}_{2,2} = \{M_1, M_2, M_3, M_4, M_5, M_{12}, M_{13}, M_{14}, M_{15}, M_{16}, M_{17}, M_{18}\}.$$

In order to separate the two classes of new and old motifs, we need first to make a compilation of all the motifs with a maximal occurrence at $[i, j]$. We assign this task to a procedure **COMPILE** which generates such motifs by computing the meets of the bite S_{ij} with every bite $S_{i'j'}$ with $(i', j') \in R_{ij}$, and then discarding those that do not have a solid character on their first row and column.

As a further intermediate step in the computation, we need to reconstruct the list of occurrences of the motifs generated by **COMPILE**. An auxiliary notion needed for this purpose is that of a core.

Definition 12 (Core). Given an image J over $\Sigma \cup \{\cdot, \cdot\}$, the *core* of J is the meet of J with the image J' that is obtained by setting to a don't care all characters in the first row and first column of J .

In particular, the core of an image can be empty.

Lemma 6. For any image M over $\Sigma \cup \{\cdot, \cdot\}$, the core of M is unique.

Proof. A straightforward consequence of the definition. ■

Lemma 7. For any $M \in R_{ij}$ with a maximal occurrence at $[i, j]$, if the core M' of M is not empty then M' has a maximal occurrence in $R_{i'j'} \neq R_{ij}$ for some $i' \geq i, j' \geq j$.

Proof. If symbols from Σ appear only on the row i and on the column j of M , then clearly M has an empty core. Suppose instead that a nonempty core M' exists for M and this is found $c > 0$ rows past row i and $d > 0$ columns to the right of column j . We see that the maximality of the occurrence of M' at $[h, k]$ with $h = i + c$ and $k = j + d$ follows from that of the occurrence of M at $[i, j]$. In fact, let M be generated by the meet of S_{ij} and some S_{fg} from $R_{i,j+1}$. Then, clearly, the meet of S_{hk} and $S_{f+c,g+d}$ must generate M' . ■

Lemma 7 gives the handle for the computation and updates of the lists of occurrences of the motifs that are maximal at $[i, j]$: assuming such lists available for $R_{i,j+1}$, the (re)construction of the list for any motif M with an occurrence at $[i, j]$ is achieved by first extracting from M its core M' and then looking for the occurrences of M' in $R_{i,j+1}$ which extend into occurrences of M . This task is assigned to a procedure **LIST**.

Upon eliminating from the rest of the motifs those found also in $\mathcal{B}_{i,j+1}$ we are left with the new motifs. This elimination is assigned to a procedure **SEPARATE**. Note that, as a by-product, **LIST** also identifies the novel motifs among the new ones. (These are motifs with only two occurrences.)

The completion of the occurrence lists for old motifs is achieved by matching one by one the elements of $\mathcal{B}_{i,j+1}$ against S_{ij} .

To summarize: old motifs can obliterate only their own submotifs; novel motifs cannot obliterate each other, but they can obliterate any other motifs; motifs of Class II that are new but not novel cannot obliterate any other motifs. We assign to a procedure **EXTRACT** the task of eliminating all the redundant motifs from the classes I and II.

5. Algorithmic implementation

In this section, we detail the mechanics of the various procedures evoked in the previous one and assess their time complexity. The latter is trivially $O(N^2)$ for the straightforward procedure **COMPILE**. (Recall that this procedure computes the meets of the bite S_{ij} with every bite $S_{i'j'}$ with $(i', j') \in R_{ij}$, and discards those that do not have a solid character on their first row and column.)

We turn then to **LIST**. Assuming inductively that the lists of occurrences were available for each of the motifs having maximal occurrences in $R_{i,j+1}$, procedure **LIST** (re)constructs the list for any such motif M with an occurrence at $[i, j]$ from the occurrences of its core M' . For this, the procedure must first extract M' . In the terms of the proof of [Lemma 7](#), it is not difficult to compute in $O(N)$ time the values c and d that separate the core from M . A more efficient way will be described next. For pedagogical reasons the computation is presented in form of preprocessing of all the autocorrelations of the image I that globally computes all c - and d -values that will be ever needed. It is left for the reader as an exercise to see that the information needed at each iteration can be carried out on-the-fly.

Preprocessing of autocorrelations. Let A be an autocorrelation and suppose to be interested in computing all the positions of the top-left corners of the meets contained in A . We call such positions *cusps* and compute them by scanning A bottom up from right to left row-wise, and deciding, for each position $[h, k]$ whether it is a cusp in overall $O(N)$, as follows. Initialize a binary vector with a 1 at position j if the last row of A has a solid character at the j th column. Let now i be the generic row of A . We scan row i from right to left while also updating the binary vector, as follows. Scan row i until the first character of σ is met at column j ; then position $[i, j]$ represents a cusp and a 1 is entered at the j th position of the binary vector. From this moment on, every position that has a solid character on row i or a 1 in the binary vector is a cusp. A solid character on row i corresponding to a 0 in the vector causes the 0 to be changed into a 1.

Clearly, the overall cost for this preprocessing of autocorrelations is $O(N^2)$. At any stage of the computation, keeping track of the top row and leftmost column generating a 1 yields the cusp of the current meet. With this preprocessing, the position of the core M' of any motif M is retrievable in constant time. In fact, M' is nothing but the cusp of the meet generating M .

Once the core M' of M has been extracted for all motifs having a maximal occurrence at $[i, j]$, **COMPILE** accesses its list of occurrences and checks which ones among them can be extended into an occurrence of M . This yields the list of occurrences of M . Checking extensions could be achieved by matching the first row and the first column of M against the corresponding rows and columns at occurrences of M' , but this would carry an unacceptable cost of $O(\sqrt{N})$, leading to $O(N^2\sqrt{N})$ for the overall cost for computing the list of each motif occurring at $[i, j]$. Following is an alternate, more efficient way. Once more, we present it in form of a global preprocessing for ease of exposition, and leave it for the reader to convert it to an incremental construction, with no penalty.

For every pair of rows i and i' in I , compute the *consensi* between row i and all the suffixes of row i' . (The same is applied to the columns of I , but we will not discuss it.) Next, compare the reverse of each such consensus with the reverse of every other row of I . For each such row q , $q \neq i$ and $q \neq i'$, store the value $\ell(i, i', h, q)$, where ℓ represents the position of row q corresponding to the rightmost mismatch on I between the reverse of q and the reverse of the consensus between row i and the h -th suffix of row i' .

Lemma 8. *The cost of computing $\ell(i, i', h, q)$ for all quadruples $\langle i, i', h, q \rangle$ is $O(N^2\sqrt{N})$.*

Proof. Computing the $O(N\sqrt{N})$ consensi between every row of I and all suffixes of every other row costs $O(N^2)$. Comparing the reverse of such consensi with each row of I now requires $O(N^2\sqrt{N})$ time. ■

With the above data, let M be a motif with a maximal occurrence at $[i, j]$, let $[i', j']$ be any occurrence of M in $R_{i,j+1}$ generating this maximal occurrence, and assume w.l.o.g. that $j' \geq j$. Finally, let the core M' of M occur at $[i + c, j + d]$. If $[f, g]$ is another occurrence of M' in $R_{i,j+1}$, then $[f - c, g - d]$ is an occurrence of M iff $\ell(i, i', j' - j, f - c) < g - d$.

Lemma 9. *The lists of occurrences of all motifs having maximal occurrences at $[i, j]$ can be computed from the corresponding lists of all the motifs $\overline{M}$ having maximal occurrences in $R_{i,j+1}$ in overall $O(N^2)$ time.*

Proof. Let M be a motif with a maximal occurrence at $[i, j]$. From Lemma 7 the core M' of M , has a maximal occurrence $R_{i,j+1}$. The preprocessing of autocorrelations make such an occurrence of M' accessible in constant time. Scanning the list of occurrences of M' costs $O(N)$, and testing for any occurrence of M' whether it expands into an occurrence of M requires constant time having computed the ℓ -values. Thus, the overall cost for computing the lists of occurrences of all the $O(N)$ motifs having maximal occurrences at $[i, j]$ is $O(N^2)$. ■

In order to propagate the invariant of Lemma 9, – that is, the hypothesis that the lists of all maximal motifs are updated at each iteration of the process – we must take care of adding $[i, j]$ also to the list of occurrences of every motif that had a maximal occurrence in $R_{i,j+1}$ though not at $[i, j]$. It is seen that an adaptation of the same technique will take care of these updates. Let M be one such motif, and consider its core M' . Then, $[i, j]$ is an occurrence of M if $[i + c, j + d]$ is an occurrence of M' , and if row and column at distance c and d from such occurrence of M' correspond, respectively, to the first row and the first column of M . Such checks need constant time, thus the overall cost charged by these $O(N^2)$ motifs to add $[i, j]$ to the maximal motifs is $O(N^2)$.

At this point, we analyze the procedure SEPARATE, which has the task of eliminating from the motifs occurring at $[i, j]$ those found also in $\mathcal{B}_{i,j+1}$. We stipulate that all the motifs in $\mathcal{B}_{i,j+1}$ are linearized, e.g., in row-major order and then stored in a trie over the alphabet $\Sigma \cup \{'.\}$, with the proviso that every row is followed by a special *eol* character. In order to identify which old motifs have an occurrence at $[i, j]$, it suffices now to similarly linearize each one of the previously found motifs occurring at $[i, j]$ and attempt inserting such motifs one by one into the trie.

At the end of this process, we know the new and old motifs. For future reference, we record the following:

Lemma 10. The cost of computing all new motifs in R_{ij} is $O(N^2)$.

Proof. This requires matching the $O(N)$ motifs occurring at $[i, j]$ in the trie, at a cost of $O(N)$ each. ■

We analyze now the procedure EXTRACT. Recall that the old motifs occurring at $[i, j]$ and the motifs that are new but not novel cannot obliterate each other. We use $\mathcal{M}$ to denote the set of all such motifs. We linearize all the novel motifs and store them in a trie over the alphabet $\Sigma \cup \{'.\}$. At the generic node v of the trie we also produce the union of all the $\mathcal{L}_{M_i}$ lists, where M_i is a motif corresponding to a subtree rooted at v . At this point, we insert each motif M in $\mathcal{M}$ into the trie. Observe that the only possible mismatches arising during the insertion must consist of a don't care in the old motif. We stop at the first such mismatch. Let μ be the branching internal node at the end of the arc corresponding to the mismatch. We claim that if the list of occurrences associated to μ contains $\mathcal{L}_M$ then M is redundant and is marked accordingly.

Lemma 11. Eliminating all the redundant motifs in $\mathcal{M}$ takes $O(N^2)$ time.

Proof. There are $O(N)$ new motifs, made each of at most N characters, whence building the trie takes $O(N^2)$. The trie itself has $O(N)$ leaves and $O(N)$ branching internal nodes, and each submotif M_i rooted at a branching internal node has $O(N)$ occurrences so the overall cost of merging the $\mathcal{L}_{M_i}$ at every such node is $O(N^2)$. The insertion attempts into the trie of each of the $O(N)$ motifs in $\mathcal{M}$ takes $O(N^2)$ overall time. All other operations are absorbed in this bound. ■

Note that the only case in which an old motif M not occurring at $[i, j]$ can be obliterated is when M is a submotif of some other motifs in Class I or II. Then, all the occurrences of M should be covered by the occurrences of such motifs that extend it. Motifs like M can be discarded during the computation of occurrences at $[i, j]$.

Theorem 5. The set of bases $\mathcal{B}_{ij}$ of irredundant motifs for all regions R_{ij} in two dimensions can be computed in $O(N^3)$.

Proof. As seen in the above discussion and lemmas, the dominant costs are charged by computing the list of occurrences of the motifs, and by the trie operations. These amount to $O(N^2)$ per iteration, yielding the bound of the claim. ■

6. Concluding remarks

In this paper, we have introduced and studied motifs arising from the autocorrelation of two-dimensional arrays of characters. As seen, the translation to two dimensions of notions and discovery paradigms previously established

in connection with strings is feasible, though not straightforward. This is in line with what already experienced in extending to two and higher dimensions other string matching tools, notably, exact searches [4,8,9] and companion notions of periodicity and repetitions [1,5,14].

It is natural to speculate as to whether also the present framework can be extended to d dimensions for general $d \geq 1$. To briefly analyze this issue, let the d -dimensional input I be given as a $n_1 \times n_2 \times \cdots \times n_d$ matrix defined on an alphabet Σ . Let the size of I be $N = n_1 \times n_2 \times \cdots \times n_d$. A d -dimensional M defined on $\Sigma \cup \{.\}$ is a k -motif if it occurs at least k times in I where occurrence has the usual meaning (as in Definition 2). Further, the notions of maximality, irredundancy, meets *etc.* translate with straightforward interpretation in d dimensions.

The central idea in our framework is that of an *autocorrelation* and its relation to the base (or irredundant) set of 2-motifs. We make the following observation.

Observation 1. *An input I of dimension d has no more than N' autocorrelations where $N' = 2^{d-1}N$.*

The corollary to this is that the size of the base in d dimension is $O(N)$, along the lines of Theorem 3. It is reasonable that also the incremental extraction algorithm may be extended on these grounds, whereby the base of an input I of dimension d can be computed in time $O(N^3)$. We leave testing this claim as an exercise for the reader.

Several interesting open questions can form the subject of further exploration. The first issue of course concerns possible improvements of the time bounds, both for incremental as well as off-line construction. Perhaps a related one concerns the possible exploitation of (on line) two-dimensional suffix trees and arrays [16] in our context. Finally, interesting open problems revolve around the discovery of motifs under rotation, a formulation that is possible to encounter in one dimension, much as it happens for the corresponding standard search problems [2,3].

Acknowledgements

The first author's work was supported in part by an IBM Faculty Partnership Award, by the Italian Ministry of University and Research under the National Projects FIRB RBNE01KNFP and PRIN "Combinatorial and Algorithmic Methods for Pattern Discovery in Biosequences", and Bi-National Project FIRB RBIN04BYZ7, and by the Research Program of the University of Padova. The third author's work was supported in part by the Italian Ministry of University and Research under the National Project PRIN "Combinatorial and Algorithmic Methods for Pattern Discovery in Biosequences".

References

- [1] A. Amir, G. Benson, Two-dimensional periodicity and its applications, in: Proceedings of the Symposium on Discrete Algorithms, SODA 1992, 1992, pp. 440–452.
- [2] A. Amir, O. Kapah, D. Tsur, Faster two dimensional pattern matching with rotations, in: Proceedings of Combinatorial Pattern Matching, CPM 2004, 2004, pp. 409–419.
- [3] A. Amir, A. Butman, M. Crochemore, G.M. Landau, M. Schaps, Two-dimensional pattern matching with rotations, Theoret. Comput. Sci. 314 (1–2) (2004) 173–187.
- [4] A. Amir, M. Farach, Two-dimensional dictionary matching, Inform. Process. Lett. 44 (5) (1992) 233–239.
- [5] A. Apostolico, V. Brimkov, Optimal discovery of repetitions in 2D, Discrete Appl. Math. 151 (1–3) (2005) 5–20.
- [6] A. Apostolico, L. Parida, Incremental paradigms of motif discovery, J. Comput. Biol. 11 (1) (2004) 15–25.
- [7] A. Apostolico, L. Parida, Compression and the wheel of fortune, in: Proceedings of IEEE Data Compression Conference, 2004, pp. 115–125.
- [8] T.P. Baker, A technique for extending rapid exact-match string matching to arrays of more than one dimension, SIAM J. Comput. 7 (1978) 533–541.
- [9] R.S. Bird, Two dimensional pattern matching, Inform. Process. Lett. 6 (1997) 168–170.
- [10] A. Blumer, J. Blumer, A. Ehrenfeucht, D. Haussler, M.T. Chen, J. Seiferas, The smallest automaton recognizing the subwords of a text, Theoret. Comput. Sci. 40 (1985) 31–55.
- [11] A. Brazma, I. Jonassen, I. Eidhammer, D. Gilbert, Approaches to the automatic discovery of patterns in biosequences, J. Comput. Biol. 5 (2) (1998) 277–304.
- [12] R. Cole, R. Hariharan, Verifying candidate matches in sparse and wildcard matching, in: Proceedings of the Symposium on Theory of Computing, STOC 2002, 2002, pp. 592–601.
- [13] M.J. Fisher, M.S. Paterson, String matching and other products, in: R.M. Karp (Ed.), Complexity of Computation, in: SIAM–AMS Proceedings, vol. 7, 1974, pp. 113–125.
- [14] Z. Galil, J.G. Park, K. Park, Three-dimensional periodicity and its application to pattern matching, SIAM J. Discrete Math. 18 (2) (2004) 362–381.

- [15] R. Grossi, N. Pisanti, M. Crochemore, M.-F. Sagot, Bases of motifs for generating repeated patterns with wild cards, *IEEE/ACM Trans. Comput. Biol. Bioinf.* 2 (1) (2005) 40–50.
- [16] J.C. Na, R. Giancarlo, K. Park, $O(n^2 \log n)$ time on-line construction of two-dimensional suffix trees, in: *Proceedings of Computing and Combinatorics, COCOON 2005*, 2005, pp. 273–282.
- [17] I. Rigoutsos, A. Floratos, L. Parida, Y. Gao, D. Platt, The emergence of pattern discovery techniques in computational biology, *J. Metabolic Eng.* 2 (3) (2000) 159–177.
- [18] H. Samet, The quadtree and related hierarchical data structures, *ACM Comput. Surv.* 16 (2) (1984) 187–260.
- [19] J.T.L. Wang, B.A. Shapiro, D.E. Shasha, *Pattern Discovery in Biomolecular Data: Tools, Techniques and Applications*, Oxford University Press, 1999.